

APRENDIZAJE DE MÁQUINAS (ACIF104) INFORME FASE 2

Nombre integrante/s: Israel Albornoz, Lilian Cifuentes, Alejandro Mayró y Osvaldo Moncada.

Curso: Aprendizaje de Máquina.

Fecha: 14/04/2026.

Contenido

I.	Introducción	3
II.	Problemática	4
III.	Metodología.....	6
a.	Análisis exploratorio de datos.....	7
b.	Selección de técnicas de aprendizaje automático	7
c.	Diseño de la arquitectura.....	7
d.	Implementación, validación y evaluación.....	7
e.	Plan de trabajo	7
IV.	Desarrollo del proyecto.....	9
a.	Análisis exploratorio de datos.....	9
b.	Selección de técnicas.....	15
1.	Naive Bayes	16
2.	Regresión Logística	16
3.	Máquina de Vectores de Soporte lineal (Linear SVM)	16
4.	Red Convolutiva Unidimensional (CNN 1D).....	17
5.	Red Bidireccional con Memoria a Corto y Largo Plazo (BiLSTM)	17
c.	Comparación de técnicas.....	18
d.	Requerimientos del proyecto	20
1.	Requerimientos Funcionales (RF)	20
2.	Requerimientos No Funcionales (RNF)	21
e.	Selección de la arquitectura	22
1.	Justificación de la selección del modelo	22
2.	Arquitectura de la red seleccionada.....	23
f.	Elaboración de modelos	24
g.	Desarrollo frontend y backend.....	25
V.	Resultados	29
1.	Análisis SHAP	30
2.	Análisis del Force Plot de SHAP	32
3.	Relación entre el uso de SHAP y el cumplimiento de requerimientos del sistema.....	34
VI.	Propuesta de mejoras	35
VII.	Código de fuente en GitHub (Anexo link).	35
VIII.	Conclusión.....	36
IX.	Bibliografía	38

I. Introducción

La propagación de noticias falsas se ha convertido en una problemática crítica dentro del ecosistema digital moderno, afectando procesos democráticos, decisiones sociales y la percepción pública de la información (Broda & Strömbäck, 2024).

En la actualidad, el crecimiento exponencial de la información digital y la masificación de las redes sociales han transformado radicalmente la manera en que las personas acceden, consumen y comparten contenido informativo. Sin embargo, esta facilidad de difusión ha generado también un aumento significativo en la propagación de noticias falsas o *fake news*, fenómeno que representa una amenaza creciente para la sociedad al afectar la opinión pública, la toma de decisiones y la confianza en los medios de comunicación. Frente a este escenario, surge la necesidad de desarrollar herramientas tecnológicas capaces de identificar y clasificar automáticamente contenidos potencialmente falsos de manera eficiente y confiable.

En este contexto, el aprendizaje de máquina se presenta como una alternativa relevante para abordar problemas de clasificación textual complejos, permitiendo detectar patrones en grandes volúmenes de datos y automatizar procesos de análisis que tradicionalmente requerirían intervención humana. Particularmente, mediante técnicas de procesamiento de lenguaje natural y algoritmos de clasificación supervisada, es posible entrenar modelos capaces de distinguir entre noticias verdaderas y falsas a partir de características lingüísticas, estructurales y semánticas presentes en el texto.

El presente proyecto tiene como objetivo desarrollar un sistema inteligente de detección de noticias falsas basado en modelos de aprendizaje automático y aprendizaje profundo, integrando distintas técnicas de clasificación para evaluar su desempeño y seleccionar la alternativa más adecuada según métricas de rendimiento, precisión y robustez. Para ello, se utiliza un conjunto de datos etiquetado compuesto por noticias reales y falsas, sobre el cual se realizan procesos de exploración, limpieza, preprocesamiento, vectorización y entrenamiento de modelos.

Asimismo, el proyecto contempla la implementación de una solución tecnológica integral compuesta por una interfaz web desarrollada en Laravel, un backend mediante API REST en Flask y un módulo de inferencia basado en Python, permitiendo construir una arquitectura funcional orientada a la predicción en tiempo real. Esta integración busca no solo validar el desempeño técnico de los modelos seleccionados, sino también demostrar su aplicabilidad práctica en un entorno real de uso.

Finalmente, el desarrollo del proyecto se estructura bajo una metodología híbrida basada en CRISP-DM y Scrum, permitiendo organizar de manera iterativa tanto la fase analítica de ciencia de datos como la construcción incremental de software. De esta forma, el presente informe documenta el proceso completo de diseño, desarrollo e implementación de la solución propuesta, desde la comprensión de la problemática hasta la construcción funcional del sistema de detección automática de fake news.

II. Problemática

En los últimos años, el crecimiento de las redes sociales y de los medios digitales ha transformado la manera en que las personas acceden, consumen y comparten información. Estas plataformas permiten una circulación inmediata de contenidos, facilitando el acceso a noticias en tiempo real. Sin embargo, esta rapidez también ha favorecido la propagación de información falsa, engañosa o no verificada. En este contexto, las *fake news* se han convertido en una problemática relevante, debido a su capacidad de influir en la opinión pública, aumentar la confusión social y afectar procesos políticos, económicos y culturales.

La gravedad del problema no radica sólo en la existencia de noticias falsas, sino en la velocidad con que se difunden y en la dificultad que tienen muchas personas para distinguir entre contenido confiable y contenido engañoso. García-Marín (2022) sostiene que en el ecosistema digital actual los fenómenos sociales se expanden con una rapidez que reduce el tiempo disponible para comprenderlos y evaluarlos críticamente. Además, plantea que la polarización puede intensificarse por la exposición constante a información alineada o contraria a las propias creencias, favoreciendo la circulación de contenidos sesgados o falsos. Esto permite entender que la desinformación no es únicamente un problema informativo, sino también social y cognitivo.

En una línea similar, Beauvais (2022) indica que la difusión de noticias falsas ha aumentado considerablemente en los últimos años, alcanzando especial visibilidad en contextos de alta sensibilidad pública, como procesos electorales y la pandemia de COVID-19. Esto muestra que la desinformación tiende a intensificarse en escenarios de incertidumbre, miedo o tensión política, donde los mensajes emocionales o simplificados se difunden con mayor facilidad. Por su parte, Broda y Strömbäck (2024), a partir de una revisión sistemática interdisciplinaria, afirman que la información falsa y engañosa puede aumentar las percepciones erróneas y la resistencia al conocimiento, generando amenazas para la salud, el bienestar, las organizaciones y la democracia. De este modo, la desinformación deja de ser sólo un problema comunicacional y pasa a constituir un riesgo social e institucional.

Asimismo, Amorós García (2018) sostiene que las *fake news* no se crean únicamente por entretenimiento, sino que suelen responder a fines económicos, políticos o ideológicos. Aunque esta referencia no pertenece a los últimos cinco años, resulta útil como apoyo conceptual para comprender que la desinformación suele responder a incentivos concretos. Esto ayuda a explicar por qué muchos contenidos apelan a emociones intensas y se difunden rápidamente antes de ser verificados.

En el plano técnico, Rojas-Rubio y Meneses-Villegas (2022) señalan que las personas no siempre son capaces de identificar correctamente si una noticia corresponde a un hecho real o a una noticia falsa. A partir de ello, justifican el uso de modelos basados en minería de datos y aprendizaje automático para abordar esta tarea. En su estudio comparan distintos algoritmos de aprendizaje automático y aprendizaje profundo aplicados a la detección de *fake news*, concluyendo que existen enfoques computacionales viables para este problema. Este antecedente resulta especialmente relevante para el presente proyecto, ya que demuestra que la clasificación automática de noticias constituye una línea de trabajo válida y técnicamente

fundamentada.

A partir de los antecedentes revisados, es posible plantear la detección de noticias falsas como una tarea de clasificación supervisada. En este enfoque, un modelo aprende a partir de ejemplos previamente etiquetados para luego asignar una categoría a nuevas instancias no vistas. En este proyecto, las categorías corresponden a noticias falsas y noticias verdaderas, utilizando como fuente principal el contenido textual de cada noticia. Esta formulación resulta adecuada, ya que permite aplicar técnicas de procesamiento de lenguaje natural para representar el texto y entrenar modelos de clasificación.

De manera complementaria, Emil (2025) realiza una revisión actualizada sobre la detección automática de fake news, destacando que los enfoques recientes han evolucionado desde métodos tradicionales de clasificación hacia arquitecturas más avanzadas, incluyendo modelos profundos y grandes modelos de lenguaje. El estudio identifica perspectivas orientadas al conocimiento, al estilo lingüístico, a la propagación y a la fuente de la desinformación, lo que refuerza la importancia de seleccionar representaciones textuales adecuadas y modelos robustos para tareas de clasificación supervisada de noticias falsas.

Además, la literatura muestra que no siempre es necesario recurrir inicialmente a arquitecturas complejas para obtener resultados relevantes. En tareas de clasificación de texto, los modelos clásicos de aprendizaje automático, combinados con técnicas de representación como TF-IDF (*Frecuencia de Término - Frecuencia Inversa de Documento*), suelen ofrecer una línea base sólida, interpretable y computacionalmente eficiente. Esto es especialmente pertinente en una primera etapa del desarrollo, donde no sólo interesa obtener un buen desempeño, sino también justificar metodológicamente la elección de técnicas, comparar resultados y reconocer limitaciones.

En relación con los datos, para el desarrollo del proyecto se propone utilizar el conjunto compuesto por los archivos Fake.csv y True.csv, ya que contiene noticias etiquetadas y texto suficiente para abordar una tarea de clasificación binaria supervisada. Este dataset resulta adecuado para una primera aproximación al problema porque permite desarrollar un flujo completo de trabajo, incluyendo exploración de datos, preprocesamiento, vectorización, entrenamiento y evaluación de modelos. Además, su estructura facilita la comparación de distintas técnicas de clasificación, lo que permite construir una línea base clara, consistente y reproducible.

No obstante, también es importante reconocer sus limitaciones. El conjunto de datos puede contener sesgos asociados a la fuente, al estilo editorial o a la temática de las noticias, por lo que los resultados deben interpretarse dentro del alcance del dataset utilizado. En consecuencia, la solución propuesta no busca determinar la verdad absoluta de una noticia, sino estimar, a partir de patrones textuales aprendidos, si una noticia se aproxima más a la categoría de falsa o verdadera.

En síntesis, Diversos estudios han demostrado que la desinformación puede influir negativamente en la salud pública, la política y la estabilidad institucional, especialmente en contextos de alta sensibilidad social como elecciones o pandemias (Beauvais, 2022; Broda & Strömbäck, 2024). la desinformación digital constituye una problemática actual y de alto impacto social, favorecida por

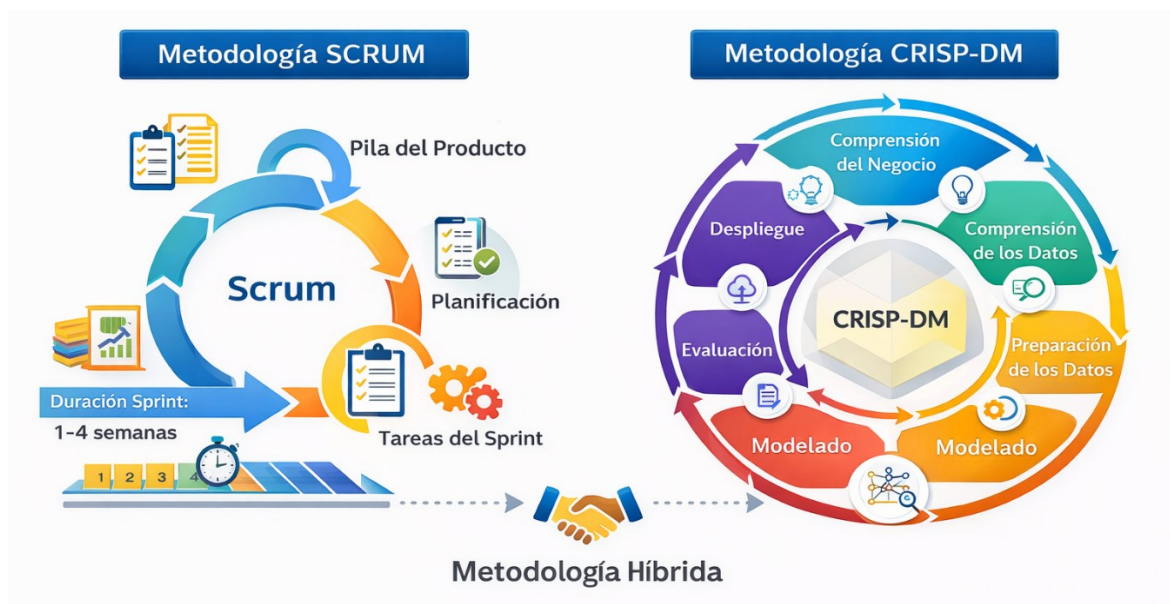
la rápida circulación de contenidos y por la dificultad de los usuarios para verificar su veracidad. En este escenario, las técnicas de aprendizaje automático surgen como una alternativa pertinente para apoyar la detección preliminar de noticias falsas. Por ello, el presente proyecto propone desarrollar un sistema de clasificación supervisada de noticias falsas y verdaderas, basado en el análisis de texto y en un conjunto de datos etiquetado, incorporando además una interfaz web y un backend con API para su implementación.

III. Metodología.

Para el desarrollo del presente proyecto se adoptó una metodología híbrida, combinando el enfoque CRISP-DM (Cross Industry Standard Process for Data Mining) con prácticas ágiles basadas en Scrum. Esta combinación resulta pertinente, ya que el proyecto integra tanto una fase de ciencia de datos orientada al entrenamiento y evaluación de modelos de clasificación, como una fase de desarrollo incremental de software para la construcción de la interfaz web, el backend y la API de predicción.

Por una parte, CRISP-DM proporciona una estructura metodológica adecuada para proyectos de aprendizaje automático, permitiendo organizar el trabajo en fases claramente definidas: comprensión del problema, comprensión y preparación de los datos, modelado, evaluación y despliegue. Este enfoque se adapta de manera natural al objetivo del proyecto, centrado en la detección automática de noticias falsas mediante modelos clásicos y de aprendizaje profundo.

Por otra parte, Scrum se incorpora como marco de trabajo ágil para la planificación, seguimiento y desarrollo incremental de los distintos componentes del sistema, incluyendo la interfaz web, la API en Flask y la integración con el modelo final seleccionado. El uso de iteraciones cortas permitió distribuir tareas entre los integrantes, validar avances parciales y ajustar decisiones técnicas conforme evolucionaba el proyecto.



En función de esta metodología híbrida, el desarrollo se estructuró en las siguientes fases:

a. Análisis exploratorio de datos

Como parte de las primeras iteraciones del proyecto, y en concordancia con las etapas iniciales de CRISP-DM, se realizó una revisión del conjunto de datos compuesto por los archivos Fake.csv y True.csv. En esta etapa se identificó la estructura de las variables, la distribución de clases, la cantidad de registros, la presencia de valores nulos y duplicados, además de la longitud de títulos y textos. También se aplicaron tareas iniciales de limpieza y normalización, lo que permitió preparar un conjunto de datos consistente para las siguientes fases de modelado.

b. Selección de técnicas de aprendizaje automático

Una vez comprendidos y preparados los datos, se avanzó hacia la fase de modelado de CRISP-DM, apoyándose en iteraciones Scrum para evaluar distintas alternativas de manera incremental. En esta etapa se decidió trabajar con modelos clásicos de aprendizaje automático, específicamente Naive Bayes, Regresión Logística y Linear SVM, junto con arquitecturas de aprendizaje profundo como CNN 1D y BiLSTM. La decisión consideró tanto la literatura reciente como los requerimientos definidos para el sistema, buscando comparar enfoques interpretables con modelos capaces de capturar patrones semánticos más complejos.

c. Diseño de la arquitectura

De forma paralela al modelado, se definió una arquitectura modular compuesta por frontend, backend y módulo de inferencia. El frontend se desarrolló en Laravel para el ingreso y visualización de noticias; el backend se implementó como una API REST en Flask para procesar solicitudes y retornar resultados; y el módulo de inferencia, desarrollado en Python, integró el modelo seleccionado junto con los artefactos necesarios para la predicción en tiempo real, como el tokenizer, max_length y el threshold de decisión.

d. Implementación, validación y evaluación

En las últimas iteraciones, se validó el funcionamiento integral del sistema, comprobando la interacción entre el frontend, el backend y el modelo de clasificación. Para ello, se realizaron pruebas con distintas noticias elaboradas manualmente, tanto verdaderas como falsas, verificando la correcta recepción de los datos, la ejecución de la predicción y la visualización del resultado. Durante este proceso, también se efectuaron ajustes de integración para asegurar una conexión estable entre la interfaz web y el modelo.

e. Plan de trabajo

Con el fin de asegurar una correcta organización, seguimiento y ejecución de las actividades asociadas al desarrollo del proyecto, se definió un plan de trabajo estructurado en etapas, alineado con la metodología híbrida adoptada entre CRISP-DM y Scrum. Este plan permitió distribuir las responsabilidades entre los integrantes del equipo, establecer plazos de ejecución y vincular cada tarea con los objetivos específicos del proyecto, facilitando el control del avance y la coordinación general durante el desarrollo.

La planificación se estructuró considerando las principales fases del ciclo de vida del proyecto: análisis del problema, exploración y preparación de datos, modelado, desarrollo de software, integración y validación. Cada actividad fue asignada a responsables específicos según su área de especialización técnica, promoviendo una distribución equilibrada de la carga de trabajo y una ejecución paralela de tareas cuando fue posible.

A continuación, se presenta la planificación general desarrollada para el proyecto:

Fase	Actividad	Responsable(s)	Duración Estimada	Relación con Objetivo del Proyecto
1	Definición de problemática y alcance del proyecto	Todos los integrantes	Semana 1	Establecer el problema a resolver y delimitar el objetivo general del sistema
2	Investigación bibliográfica y revisión de literatura	Israel Albornoz / Lilian Cifuentes	Semana 1 – 2	Fundamentar teóricamente el uso de aprendizaje automático en fake news
3	Selección y análisis inicial del dataset	Alejandro Mayrú / Osvaldo Moncada	Semana 2	Identificar la fuente de datos adecuada para entrenamiento del modelo
4	Limpieza, preprocesamiento y análisis exploratorio de datos (EDA)	Alejandro Mayrú	Semana 2 – 3	Preparar los datos para modelado e identificar patrones relevantes
5	Diseño de arquitectura del sistema	Osvaldo Moncada / Alejandro Mayrú	Semana 3	Definir la estructura frontend-backend-modelo
6	Implementación de modelos de Machine Learning clásicos	Israel Albornoz	Semana 3 – 4	Evaluar modelos base para clasificación de noticias
7	Implementación de modelos Deep Learning (CNN, LSTM, BiLSTM)	Alejandro Mayrú	Semana 4 – 5	Evaluar arquitecturas profundas para mejorar precisión predictiva
8	Comparación de métricas y selección preliminar de modelo	Todos los integrantes	Semana 5	Seleccionar mejor técnica según accuracy, precision, recall y F1-score
9	Desarrollo de frontend en Laravel	Osvaldo Moncada	Semana 5 – 6	Construir interfaz gráfica de interacción para el usuario
10	Desarrollo de backend/API REST en Flask	Alejandro Mayrú	Semana 6	Implementar servicio de inferencia y comunicación con frontend
11	Integración frontend-backend-modelo	Osvaldo Moncada / Alejandro Mayrú	Semana 6 – 7	Consolidar funcionamiento completo del sistema
12	Validación funcional y pruebas del sistema	Todos los integrantes	Semana 7	Verificar correcto funcionamiento integral de la solución
13	Documentación técnica y elaboración de informe final	Todos los integrantes	Semana 7 – 8	Formalizar resultados, metodología y desarrollo del proyecto

Cronograma Gantt Proyecto Fake News

Actividad	S1	S2	S3	S4	S5	S6	S7	S8
Definición problemática	■							
Investigación bibliográfica	■	■						
Selección dataset		■						
EDA y limpieza datos		■	■					
Diseño arquitectura			■					
Modelos clásicos ML			■	■				
Deep Learning				■	■			
Comparación métricas					■			
Frontend Laravel					■	■		
Backend Flask/API					■	■		
Integración sistema						■	■	
Validación pruebas							■	
Documentación final							■	■

Asignación de responsabilidades

La distribución de tareas fue realizada considerando las competencias técnicas y áreas de apoyo de cada integrante, permitiendo optimizar la ejecución paralela de actividades y mejorar la eficiencia del equipo de trabajo:

- **Alejandro Mayró:** responsable principal del procesamiento de datos, análisis exploratorio, entrenamiento de modelos de aprendizaje profundo y desarrollo backend.
- **Osvaldo Moncada:** responsable del diseño arquitectónico del sistema, desarrollo frontend e integración general de componentes.
- **Israel Albornoz:** Responsable de investigación bibliográfica, fundamentación metodológica y entrenamiento de modelos clásicos de machine learning.
- **Lilian Cifuentes:** Responsable de apoyo en investigación documental, validación bibliográfica y revisión de documentación técnica.

IV. Desarrollo del proyecto.

a. Análisis exploratorio de datos

Para el desarrollo del sistema de detección de noticias falsas se seleccionó el conjunto de datos Fake and Real News Dataset, disponible en Kaggle. Este dataset fue elegido por ser ampliamente utilizado en tareas de clasificación binaria de texto, especialmente en problemas de detección de fake news. Su estructura resulta adecuada para el presente proyecto, ya que contiene noticias previamente etiquetadas en dos categorías claramente definidas: falsas y verdaderas.

El conjunto de datos se encuentra dividido en dos archivos: Fake.csv, que reúne 23.481 noticias falsas, y True.csv, que contiene 21.417 noticias verdaderas. Cada registro incluye cuatro atributos principales: title (título de la noticia), text (contenido textual), subject (categoría temática) y date (fecha de publicación). Esta composición permitió contar con una base suficiente y pertinente para realizar las etapas de exploración, preprocesamiento, modelado y evaluación del sistema.

Dimensiones Fake.csv: (23481, 4)

Dimensiones True.csv: (21417, 4)

Columnas Fake: ['title', 'text', 'subject', 'date']
Columnas True: ['title', 'text', 'subject', 'date']

La selección de este conjunto de datos se justifica por varias razones. En primer lugar, presenta un volumen suficiente de observaciones para entrenar y evaluar modelos de aprendizaje automático y aprendizaje profundo. En segundo lugar, incluye tanto el título como el cuerpo completo de la noticia, lo que permite aprovechar información textual rica y construir

representaciones más robustas. Finalmente, la existencia de una etiqueta binaria clara facilita la formulación del problema como una tarea de clasificación supervisada.

Tras cargar ambos archivos en Pandas, se comprobó que compartieran la misma estructura de columnas para permitir su integración en un único conjunto de datos. Posteriormente, se añadió la variable objetivo label, asignando el valor 1 a las noticias falsas y 0 a las verdaderas. Esta codificación permitió consolidar ambos archivos en una sola base de datos, conservando la identificación de la clase correspondiente para cada registro.

```
# =====
# 4. Crear etiqueta de clase
# 1 = fake
# 0 = true
# =====
df_fake["label"] = 1
df_true["label"] = 0

print(df_fake["label"].value_counts())
print(df_true["label"].value_counts())

label
1    23481
Name: count, dtype: int64
label
0    21417
Name: count, dtype: int64
```

Después de integrar ambos datasets, se llevó a cabo una fase de revisión de calidad de datos. En esta etapa se inspeccionaron los tipos de datos de cada columna, la presencia de valores nulos, la completitud de las variables y la distribución de clases. Asimismo, se verificó la existencia de títulos o textos vacíos, junto con la detección de registros duplicados exactos y duplicados parciales basados en la combinación title + text.

```
Títulos vacíos: 0
Textos vacíos: 631

Textos vacíos por clase:
label
1    630
0     1
Name: count, dtype: int64
```

```
Duplicados exactos: 209
Duplicados por title + text: 5793
```

Se generó un resumen de calidad por variable, considerando el tipo de dato, la presencia de valores nulos, la cantidad de registros no nulos y el número de valores únicos por columna. Los resultados muestran que el dataset presenta completitud total en todas sus variables, sin registros nulos, lo que confirma una estructura consistente para las etapas posteriores de limpieza y análisis.

	Variable	Descripción	Tipo
0	title	Título de la noticia	Texto
1	text	Contenido textual completo de la noticia	Texto
2	subject	Categoría temática de la noticia	Catórica
3	date	Fecha de publicación	Texto/fecha
4	label	Clase objetivo (0 = verdadera, 1 = falsa)	Binaria

Como complemento a la revisión de calidad, se realizó una validación de correctitud sobre variables clave, inspeccionando registros con títulos extremadamente cortos, textos con muy pocas palabras y fechas no convertibles. Esta revisión permitió identificar casos atípicos asociados principalmente a URLs incrustadas, textos incompletos y formatos inconsistentes en la columna date, elementos que fueron considerados posteriormente en la fase de limpieza del dataset.

- Registros con title muy corto (≤ 2 palabras):

	title	text
9358	https://100percentfedup.com/served-roy-moore-vietnamletter-veteran-sets-record-straight-honorable-decent-respectable...	https://100percentfedup.com/served-roy-moore-vietnamletter-veteran-sets-record-straight-honorable-decent-respectable...
15507	https://100percentfedup.com/video-hillary-asked-about-trump-i-just-want-to-eat-some-pie/	https://100percentfedup.com/video-hillary-asked-about-trump-i-just-want-to-eat-some-pie/

- Registros con text muy corto (≤ 5 palabras):

	title	text	label
9358	https://100percentfedup.com/served-roy-moore-vietnamletter-veteran-sets-record-straight-honorable-decent-respectable...	https://100percentfedup.com/served-roy-moore-vietnamletter-veteran-sets-record-straight-honorable-decent-respectable...	1
10923	TAKE OUR POLL: Who Do You Think President Trump Should Pick To Replace James Comey?		1

- Ejemplos de fechas inválidas o no convertibles:

	date
9358	https://100percentfedup.com/served-roy-moore-vietnamletter-veteran-sets-record-straight-honorable-decent-respectable...
15507	https://100percentfedup.com/video-hillary-asked-about-trump-i-just-want-to-eat-some-pie/
15508	https://100percentfedup.com/12-yr-old-black-conservative-whose-video-to-obama-went-viral-do-you-really-love-america-...

Con el objetivo de comprender la distribución temática del dataset, se analizó la variable subject en función de la clase asignada. Esta revisión permitió identificar las principales categorías presentes en las noticias verdaderas y falsas, observándose diferencias claras en la procedencia temática de cada grupo. Mientras las noticias verdaderas se concentran principalmente en categorías como politicsNews y worldnews, las noticias falsas presentan una mayor presencia en News, politics y left-news.

Distribución de subject por clase:

label	0	1
subject		
Government News	0	1570
Middle-east	0	778
News	0	9050
US_News	0	783
left-news	0	4459
politics	0	6841
politicsNews	11272	0
worldnews	10145	0

Como parte del preprocesamiento inicial, se eliminaron los registros con texto vacío y los duplicados identificados en la combinación title + text. Luego, se aplicó una limpieza básica del texto, convirtiéndolo a minúsculas, eliminando URLs, caracteres no alfabéticos y espacios en blanco innecesarios. A partir de este proceso se generaron las variables title_clean y text_clean, que serán utilizadas en las etapas posteriores del proyecto.

```

Fechas no convertibles: 10
Ejemplos de fechas inválidas:
9358 https://100percentfedup.com/served-roy-moore-vietnamletter-veteran-sets-record-straight-honorable-decent-respectable...
date

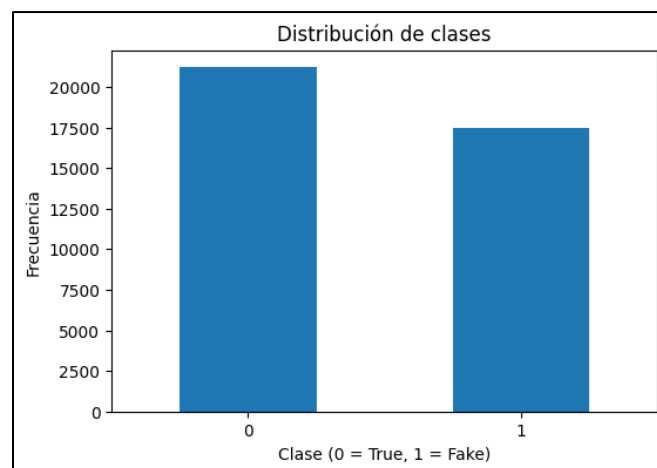
```

```

Registros antes: 44267
Registros después: 38658
Duplicados eliminados: 5609

```

Una vez finalizado el proceso de limpieza y eliminación de duplicados, se analizó nuevamente la distribución de clases del conjunto de datos resultante. Como se observa, la base mantiene una distribución relativamente equilibrada entre noticias verdaderas (clase 0) y falsas (clase 1), aunque con una ligera mayor presencia de la clase verdadera.



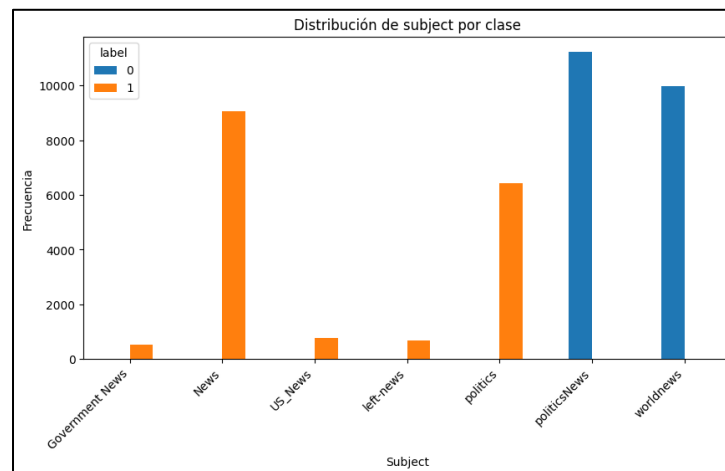
Se generó un resumen estadístico de las variables numéricas derivadas del texto, específicamente la longitud de títulos y contenidos en términos de palabras y caracteres. Este análisis permitió identificar la distribución general del tamaño de las noticias, observándose que los títulos presentan una longitud promedio de aproximadamente 12 palabras, mientras que el

cuerpo de la noticia alcanza un promedio cercano a 408 palabras. Asimismo, la amplitud entre valores mínimos y máximos evidenció la existencia de noticias considerablemente más extensas que otras, aspecto relevante para definir estrategias de tokenización, padding y longitud máxima de entrada en las etapas posteriores del modelado.

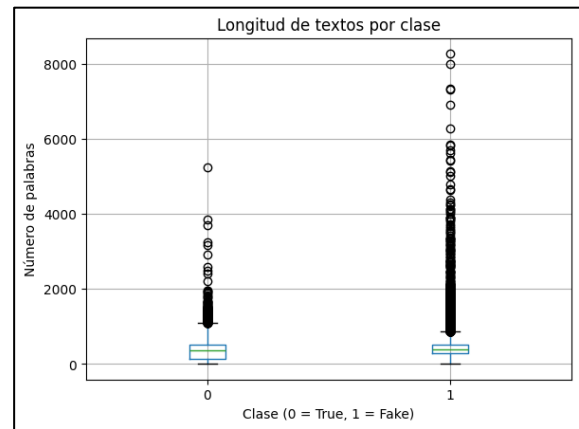
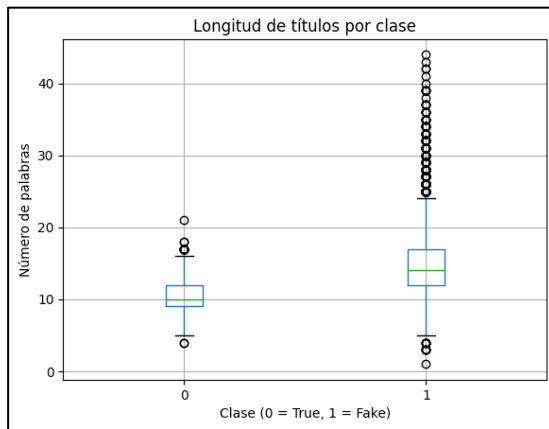
Resumen estadístico de variables numéricas:

	title_len	text_len	title_char_len	text_char_len
count	38658.000000	38658.000000	38658.000000	38658.000000
mean	12.451963	408.421776	76.107300	2454.208081
std	3.895156	319.216064	21.945306	1936.865674
min	1.000000	1.000000	8.000000	4.000000
25%	10.000000	219.000000	62.000000	1316.000000
50%	12.000000	374.000000	71.000000	2226.000000
75%	14.000000	518.000000	85.000000	3095.000000
max	44.000000	8281.000000	279.000000	51793.000000

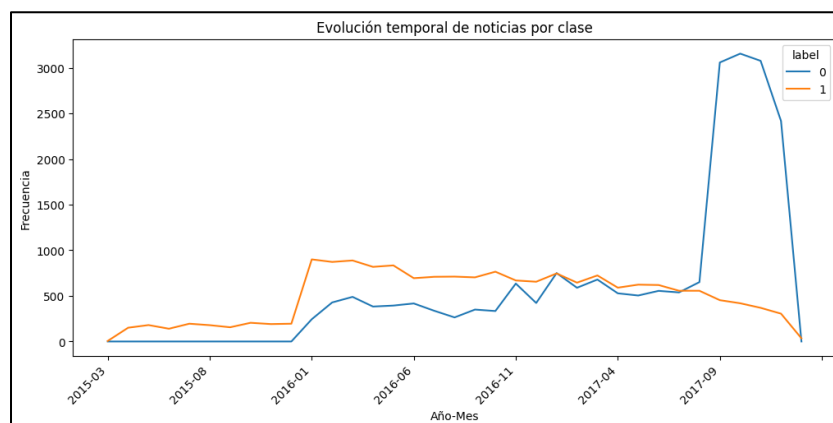
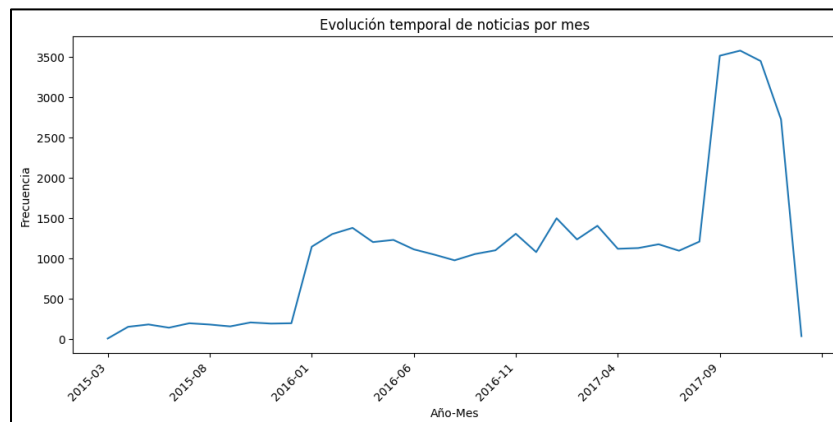
Con el fin de profundizar en el comportamiento de la variable subject, se analizó su distribución en función de la clase objetivo. Como se observa, existe una separación muy marcada entre algunas categorías temáticas y la clase asignada: politicsNews y worldnews aparecen exclusivamente en noticias verdaderas, mientras que categorías como News, politics y left-news se concentran principalmente en noticias falsas. Este patrón evidencia un posible sesgo temático o fuga de información, aspecto que debe considerarse cuidadosamente en la etapa de modelado para evitar que el modelo aprenda señales demasiado dependientes de la categoría.



Se comparó la longitud promedio de títulos y textos por clase, complementando el análisis con diagramas de caja para ambas variables. Los resultados muestran que las noticias falsas (clase 1) presentan, en promedio, títulos y textos más largos, además de una mayor dispersión y presencia de valores atípicos que las noticias verdaderas. Esto sugiere que la longitud del contenido puede aportar información útil para la clasificación.



Finalmente, se incorporó un análisis temporal del dataset a partir de la variable date, con el fin de observar la evolución mensual de las noticias y su comportamiento según la clase asignada. Los resultados muestran un incremento progresivo en la cantidad de registros, con picos más marcados hacia finales de 2017, lo que sugiere una mayor concentración de actividad informativa en ese periodo. Al desagregar por clase, se observa que en los primeros meses predominan las noticias falsas, mientras que hacia el final del intervalo aumenta considerablemente la presencia de noticias verdaderas. Este comportamiento evidencia variaciones en la composición temporal del dataset, aspecto que debe considerarse en el modelado, ya que podría influir en los patrones aprendidos por los modelos.



En conjunto, el análisis exploratorio permitió identificar patrones temáticos, diferencias estructurales entre clases, posibles sesgos del dataset y características relevantes del texto, proporcionando una base sólida para la selección de técnicas y la configuración del proceso de modelado.

b. Selección de técnicas

La detección automática de noticias falsas se aborda como un problema de clasificación supervisada de texto, en el cual un modelo aprende, a partir de ejemplos previamente etiquetados, a diferenciar entre noticias verdaderas y falsas. La literatura reciente muestra que esta tarea ha sido abordada tanto con modelos clásicos de aprendizaje automático como con enfoques más complejos de aprendizaje profundo. Sin embargo, distintas revisiones coinciden en que la selección de una técnica adecuada depende del tipo de datos disponible, de la representación textual utilizada y de los requerimientos del sistema, tales como interpretabilidad, tiempo de respuesta y facilidad de integración. En particular, las revisiones de Capuano et al. (2023) y Hu et al. (2025) permiten situar la detección de *fake news* basada en contenido como una tarea de clasificación, donde los modelos clásicos siguen siendo una alternativa válida con representaciones como TF-IDF.

En el contexto del presente proyecto, la selección de técnicas debe responder a las características del problema definidas en las secciones anteriores. Por una parte, el conjunto de datos elegido contiene texto etiquetado, lo que favorece el uso de técnicas supervisadas. Por otra, los requerimientos del proyecto exigen una solución con buen rendimiento, capaz de integrarse a una interfaz web con backend, ofrecer tiempos de respuesta breves y mantener cierto grado de explicabilidad. En este escenario, resulta metodológicamente adecuado comenzar con modelos clásicos de aprendizaje automático que permitan construir una línea base clara, reproducible y comparativa, para luego contrastar su desempeño con arquitecturas de aprendizaje profundo orientadas al procesamiento secuencial y local del texto. Esta estrategia permite evaluar no solo la efectividad de representaciones tradicionales como TF-IDF, sino también la capacidad de modelos neuronales para capturar patrones semánticos más complejos. La decisión es coherente con revisiones recientes que destacan el valor de comparar enfoques clásicos y profundos en tareas de detección de *fake news* basadas en contenido textual (Capuano et al., 2023; Hu et al., 2025).

A partir de estos antecedentes, se proponen como técnicas candidatas de evaluación tanto modelos clásicos de aprendizaje automático como arquitecturas de aprendizaje profundo. En la primera categoría se consideran Naive Bayes, Regresión Logística y Máquina de Vectores de Soporte lineal (Linear SVM), mientras que en la segunda se incorporan una red convolucional unidimensional (CNN 1D), una red bidireccional con memoria a corto y largo plazo (BiLSTM) y una red con memoria a corto y largo plazo (LSTM). Esta selección permite comparar enfoques de distinta complejidad y evaluar su desempeño mediante métricas estándar como accuracy, precisión, recall y puntuación F1, en concordancia con los requerimientos definidos para el proyecto.

1. Naive Bayes

Naive Bayes es un clasificador probabilístico ampliamente utilizado en tareas de clasificación de texto por su simplicidad, rapidez de entrenamiento y bajo costo computacional. En problemas de noticias falsas, esta técnica continúa siendo considerada una línea base útil, especialmente cuando el texto se representa mediante frecuencias o ponderaciones de términos. Estudios recientes la incluyen dentro de los algoritmos evaluados para detección de *fake news*, junto con otros modelos clásicos, precisamente por su capacidad para ofrecer resultados razonables con una implementación sencilla (Imbwaga et al., 2022).

Su principal fortaleza, en el marco de este proyecto, radica en que puede satisfacer adecuadamente requerimientos de rendimiento y escalabilidad, permitiendo entrenar y predecir con tiempos reducidos. No obstante, su limitación más conocida es la fuerte suposición de independencia entre características, lo que puede reducir su capacidad para modelar relaciones semánticas más complejas presentes en los textos periodísticos. Por ello, Naive Bayes resulta pertinente como técnica de referencia inicial, aunque no necesariamente como la opción final más robusta (Capuano et al., 2023).

2. Regresión Logística

La Regresión Logística es una técnica de clasificación binaria ampliamente utilizada en procesamiento de lenguaje natural cuando los textos se representan mediante TF-IDF o *Bag of Words*. Su principal ventaja es que ofrece un equilibrio adecuado entre desempeño, estabilidad e interpretabilidad. Esta última característica es especialmente importante para el presente proyecto, ya que uno de los requerimientos no funcionales definidos corresponde a la explicabilidad del sistema. En este sentido, la Regresión Logística permite analizar con relativa claridad el peso de ciertas características en la decisión del modelo, lo que la hace adecuada para sistemas que deben entregar resultados comprensibles al usuario (Capuano et al., 2023).

Además, existen trabajos recientes donde la Regresión Logística obtiene resultados competitivos en tareas de detección de *fake news*. Por ejemplo, Passi y Shah (2022) reportan que, utilizando características TF-IDF sobre datos de Twitter, la mayor exactitud fue obtenida por la Regresión Logística, con un valor de 94,79 %. Asimismo, Sudhakar y Kaliyamurthie (2024) la incluyen entre las técnicas comparadas en su estudio, alcanzando una exactitud de 95 %. Estos antecedentes respaldan su pertinencia como candidata fuerte dentro del conjunto de modelos a evaluar.

3. Máquina de Vectores de Soporte lineal (Linear SVM)

Las Máquinas de Vectores de Soporte, particularmente en su variante lineal, son una de las técnicas más utilizadas en clasificación de texto debido a su buen comportamiento en espacios de alta dimensionalidad. Dado que la representación TF-IDF genera vectores dispersos y de gran tamaño, SVM lineal resulta especialmente adecuada para este tipo de problemas. La literatura reciente la sigue considerando una alternativa robusta en detección de noticias falsas, tanto en estudios comparativos como en propuestas centradas en modelos clásicos (Imbwaga et al., 2022; Sudhakar & Kaliyamurthie, 2024).

En términos prácticos, su principal ventaja para este proyecto es la capacidad de ofrecer un muy buen desempeño sin requerir la complejidad de modelos profundos. Sudhakar y Kaliyamurthie (2024) informan que SVM alcanzó 98 % de exactitud en su estudio, superando a la Regresión Logística en ese conjunto experimental. De forma similar, otros trabajos recientes continúan incluyendo SVM como una de las técnicas base para sistemas de detección textual de *fake news*, precisamente por su robustez frente a datos de alta dimensionalidad. Como limitación, su interpretabilidad directa es menor que la de la Regresión Logística, aunque sigue siendo una técnica altamente pertinente cuando se prioriza desempeño y estabilidad.

4. Red Convolucional Unidimensional (CNN 1D)

La red convolucional unidimensional (CNN 1D) corresponde a una arquitectura de aprendizaje profundo ampliamente utilizada en clasificación de texto, debido a su capacidad para detectar patrones locales relevantes dentro de las secuencias, tales como combinaciones frecuentes de palabras, expresiones y estructuras lingüísticas asociadas a noticias falsas. A diferencia de los modelos clásicos basados en TF-IDF, esta técnica permite identificar patrones directamente desde el texto tokenizado, reduciendo la dependencia de una selección manual de características.

En el contexto del presente proyecto, la CNN 1D se incorpora como una técnica de comparación avanzada frente a los modelos clásicos, permitiendo evaluar si la extracción automática de patrones locales mejora la capacidad predictiva del sistema. Entre sus principales ventajas destacan su buen equilibrio entre precisión, velocidad de entrenamiento y eficiencia en inferencia, lo que la convierte en una alternativa especialmente adecuada para una futura integración en el backend del sistema web.

5. Red Bidireccional con Memoria a Corto y Largo Plazo (BiLSTM)

La arquitectura BiLSTM (Bidirectional Long Short-Term Memory) corresponde a un modelo de aprendizaje profundo orientado al procesamiento secuencial del texto, capaz de analizar la información en ambas direcciones de la secuencia: desde el inicio hacia el final y viceversa. Esta característica permite capturar dependencias contextuales más complejas entre palabras, lo que resulta especialmente útil en textos periodísticos donde el significado depende del orden y de relaciones semánticas más extensas.

En este proyecto, la BiLSTM se incorpora como una técnica avanzada de comparación frente a CNN 1D y a los modelos clásicos, con el objetivo de determinar si la comprensión bidireccional del contexto textual mejora la detección de noticias falsas. Su incorporación permite ampliar la evaluación experimental del sistema, comparando enfoques tradicionales basados en representaciones vectoriales con modelos neuronales capaces de captar relaciones semánticas más complejas a partir del texto.

6. Red con Memoria a Corto y Largo Plazo (LSTM)

La arquitectura LSTM (Long Short-Term Memory) corresponde a un modelo de aprendizaje profundo especializado en el procesamiento de secuencias de texto, diseñado para capturar dependencias entre palabras a lo largo de la secuencia. Su principal ventaja frente a modelos

clásicos es la capacidad de retener información contextual relevante, incluso cuando esta se encuentra separada por varias posiciones dentro del texto.

En el contexto de este proyecto, la LSTM se incorpora como una tercera arquitectura de aprendizaje profundo para comparar su desempeño frente a CNN 1D y BiLSTM, evaluando si el modelado secuencial unidireccional resulta suficiente para la detección de noticias falsas. Su inclusión permite ampliar la fase experimental y contrastar distintos niveles de complejidad en la comprensión semántica del texto.

c. Comparación de técnicas.

En primer lugar, todas las técnicas seleccionadas son apropiadas para trabajar con texto procesado, ya sea mediante representaciones vectoriales tradicionales como TF-IDF o mediante secuencias tokenizadas utilizadas por arquitecturas neuronales. En segundo lugar, sus costos computacionales y capacidades predictivas permiten evaluar distintas alternativas en función del requisito de rendimiento del sistema y de su futura integración en una arquitectura compuesta por interfaz web y backend. En tercer lugar, permiten construir una comparación progresiva entre modelos clásicos y de aprendizaje profundo, condición relevante para una etapa experimental orientada a seleccionar la mejor alternativa para la solución final (Capuano et al., 2023; Hu et al., 2025).

Desde una perspectiva comparativa, Naive Bayes se destaca por su simplicidad, rapidez y facilidad de implementación, por lo que resulta útil como línea base inicial (Imbwaga et al., 2022). La Regresión Logística ofrece un mejor equilibrio entre desempeño e interpretabilidad, siendo especialmente pertinente por el requisito de explicabilidad del sistema (Capuano et al., 2023). Linear SVM, por su parte, aparece como una técnica robusta cuando se busca maximizar el desempeño en clasificación textual con representaciones TF-IDF, manteniéndose como una referencia sólida en la literatura reciente (Sudhakar & Kaliyamurthie, 2024). En cuanto a las arquitecturas de aprendizaje profundo, CNN 1D permite capturar patrones locales relevantes dentro de las secuencias de texto; BiLSTM incorpora una comprensión bidireccional del contexto, lo que puede favorecer una mejor interpretación semántica de noticias complejas; y LSTM aporta la capacidad de modelar dependencias secuenciales a lo largo del contenido. Esta comparación resulta coherente con revisiones recientes que recomiendan contrastar modelos clásicos con arquitecturas profundas en tareas de detección de fake news basadas en contenido textual (Capuano et al., 2023; Hu et al., 2025).

Aunque los enfoques de aprendizaje profundo suelen implicar un mayor costo de entrenamiento y ajuste de hiperparámetros, su incorporación en esta etapa experimental resulta metodológicamente pertinente, ya que permite evaluar si la extracción automática de patrones semánticos mejora significativamente la clasificación frente a técnicas clásicas. En consecuencia, la propuesta metodológica del proyecto considera estas seis técnicas como candidatas de evaluación, justificadas tanto por la evidencia en trabajos previos como por su ajuste a los requerimientos funcionales y no funcionales definidos para la solución propuesta (Fedushko et al., 2024; Hu et al., 2025).

Tabla 1. Comparación de técnicas candidatas para la detección de noticias falsas

Técnica	Fortalezas	Limitaciones	Ajuste al proyecto
Naive Bayes	Rápido, simple y de bajo costo computacional	Supone independencia entre términos	Útil como línea base inicial y para cumplir requisitos de rendimiento
Regresión Logística	Buen equilibrio entre desempeño e interpretabilidad	Menor capacidad para capturar relaciones no lineales complejas	Pertinente por el requisito de explicabilidad
Linear SVM	Robusto en texto de alta dimensionalidad y buen desempeño con TF-IDF	Menor interpretabilidad directa	Adecuado cuando se prioriza desempeño y estabilidad
CNN 1D	Detecta patrones locales automáticamente y ofrece buen desempeño en texto	Requiere mayor costo de entrenamiento que modelos clásicos	Adecuada para comparar aprendizaje profundo con buena eficiencia
BiLSTM	Captura dependencias contextuales bidireccionales y relaciones semánticas complejas	Mayor tiempo de entrenamiento y complejidad	Pertinente para evaluar comprensión secuencial profunda del texto
LSTM	Captura dependencias secuenciales y contexto textual	Mayor costo computacional que modelos clásicos	Adecuada para evaluar el aporte del modelado secuencial frente a CNN 1D y BiLSTM

d. Requerimientos del proyecto

1. Requerimientos Funcionales (RF)

- **RF01 - Ingreso de texto:** El sistema debe permitir al usuario ingresar, a través de una interfaz web, el contenido textual de una noticia para su posterior análisis (título y contenido).
- **RF02 - Comunicación entre interfaz y backend:** El sistema debe enviar la noticia ingresada desde la interfaz web hacia una API encargada de procesar la solicitud y gestionar la comunicación con el modelo de aprendizaje automático.
- **RF03 - Preprocesamiento y representación del texto:** El sistema debe realizar automáticamente la limpieza, normalización y vectorización del texto ingresado, con el fin de convertirlo en un formato apto para su análisis por el modelo de aprendizaje automático.
- **RF04 - Procesamiento y comparación:** El sistema debe analizar el texto procesado y contrastar sus características lingüísticas y semánticas con los patrones aprendidos a partir de noticias verdaderas y falsas durante el entrenamiento.
- **RF05 - Clasificación de veracidad:** El sistema de clasificación debe categorizar la información ingresada mediante una etiqueta binaria (Verdadera/Falsa), acompañada de un indicador de nivel de confianza o probabilidad.
- **RF06 - Visualización del resultado:** El sistema debe mostrar al usuario el resultado final de la clasificación de la noticia analizada, indicando si fue clasificada como verdadera o falsa, junto con su nivel de confianza.
- **RF07 - Interfaz de notificación:** El sistema debe desplegar una alerta visual clara al usuario cuando el resultado del análisis identifique una noticia como potencialmente falsa, proporcionando una advertencia inmediata.
- **RF08 - Comparación de modelos de clasificación:** El sistema debe permitir la ejecución y comparación de distintas técnicas de aprendizaje automático, evaluando su desempeño en la tarea de clasificación mediante métricas como accuracy, precisión, recall y puntuación F1, con el fin de seleccionar el modelo más adecuado.

2. Requerimientos No Funcionales (RNF)

- **RNF01 - Rendimiento (Velocidad):** El tiempo de respuesta del sistema para el análisis de una noticia estándar no debe exceder los 2 a 3 segundos, de modo que se garantice una interacción fluida con el usuario.
- **RNF02 – Integridad:** El sistema debe resguardar la integridad de los datos de entrada y de los resultados generados, evitando alteraciones no autorizadas durante el procesamiento.
- **RNF03 - Confiabilidad:** El sistema debe mantener un funcionamiento estable y entregar resultados consistentes ante distintos tipos de redacción y estructura textual.
- **RNF04 - Usabilidad:** La interfaz debe ser intuitiva y comprensible para usuarios sin conocimientos técnicos, permitiendo interpretar fácilmente el resultado entregado por el sistema.
- **RNF05 - Explicabilidad (XAI):** El sistema no solo debe decir "es falso", sino dar una breve razón (ej: "uso excesivo de lenguaje emocional" o "falta de fuentes verificables"), lo que reduce la "resistencia al conocimiento" que mencionan Broda y Strömbäck (2024).
- **RNF06 - Escalabilidad:** El sistema debe ser capaz de adaptarse a conjuntos de datos de mayor tamaño y a un aumento en la cantidad de consultas, sin requerir cambios estructurales significativos.
- **RNF07 - Monitoreabilidad:** El sistema debe contar con mecanismos de registro y seguimiento del desempeño del modelo, permitiendo identificar variaciones en métricas como accuracy, precisión, recall y puntuación F1 ante nuevos datos.
- **RNF08 - Compatibilidad:** El sistema debe ser compatible con navegadores web modernos y permitir la comunicación mediante protocolos HTTP/JSON entre la interfaz y la API.
- **RNF09 - Mantenibilidad:** El sistema debe estar organizado de forma modular, separando interfaz, backend y modelo de clasificación, para facilitar futuras mejoras y actualizaciones.
- **RNF10 - Calidad del modelo:** El sistema de clasificación debe alcanzar métricas de desempeño superiores al 95% en F1-score y accuracy sobre el conjunto de prueba, garantizando una clasificación confiable.

e. Selección de la arquitectura

Con el objetivo de determinar la arquitectura más adecuada para la solución propuesta, se realizó una evaluación comparativa entre los distintos modelos de clasificación considerados durante la fase experimental del proyecto, incluyendo técnicas clásicas de aprendizaje automático (Naive Bayes, Regresión Logística y Linear SVM) y arquitecturas de aprendizaje profundo (CNN 1D, LSTM y BiLSTM). La selección final se efectuó considerando el comportamiento de convergencia durante el entrenamiento, el error de validación, las métricas de desempeño obtenidas y el cumplimiento de los requerimientos funcionales y no funcionales previamente definidos.

La evaluación experimental permitió observar que, si bien los modelos clásicos presentaron un buen desempeño inicial y tiempos de entrenamiento considerablemente menores, las arquitecturas de aprendizaje profundo mostraron una mejor capacidad para capturar patrones secuenciales y relaciones semánticas complejas dentro del texto, aspecto especialmente relevante en tareas de clasificación de noticias donde el contexto y el orden de las palabras influyen significativamente en la interpretación del contenido.

Entre los modelos evaluados, la arquitectura Bidirectional Long Short-Term Memory (BiLSTM) presentó el mejor equilibrio general entre precisión predictiva, estabilidad de convergencia y capacidad de generalización sobre datos no vistos, razón por la cual fue seleccionada como arquitectura principal del sistema de clasificación. Esta elección se encuentra alineada con la literatura reciente, la cual destaca la eficacia de redes recurrentes bidireccionales en tareas de procesamiento de lenguaje natural debido a su capacidad para analizar dependencias contextuales en ambas direcciones de una secuencia textual.

1. Justificación de la selección del modelo

La arquitectura BiLSTM fue seleccionada debido a que presentó las mejores métricas globales de desempeño dentro del conjunto de modelos evaluados, destacándose particularmente en precisión, recall y F1-score, métricas fundamentales para el presente proyecto dada la necesidad de minimizar tanto falsos positivos como falsos negativos en la detección de noticias falsas.

Asimismo, durante el proceso de entrenamiento se observó una convergencia estable de la función de pérdida (*loss*), sin evidencias significativas de sobreajuste, manteniendo un comportamiento consistente entre el error de entrenamiento y validación. Este comportamiento sugiere una adecuada capacidad de generalización del modelo, permitiendo su utilización en escenarios prácticos fuera del conjunto de entrenamiento.

Además, la arquitectura seleccionada cumplió satisfactoriamente con el requerimiento no funcional de calidad del modelo (RNF10), alcanzando valores superiores al 95 % en métricas de clasificación sobre el conjunto de prueba, así como con el requerimiento de rendimiento (RNF01), manteniendo tiempos de inferencia compatibles con la integración en tiempo real dentro del backend del sistema.

2. Arquitectura de la red seleccionada

La arquitectura implementada se compone de las siguientes capas:

- **Capa de entrada:** La capa de entrada recibe como datos secuencias tokenizadas correspondientes al texto procesado de cada noticia, previamente transformadas mediante tokenización y ajustadas a una longitud máxima fija (*max_length*) mediante padding. Esta configuración permite uniformar la dimensionalidad de entrada para el procesamiento secuencial de la red neuronal.
- **Capa Embedding:** Se incorpora una capa de *Embedding* encargada de transformar cada token en un vector denso de representación semántica. Esta capa permite mapear palabras similares en espacios vectoriales cercanos, mejorando la capacidad del modelo para identificar relaciones contextuales dentro del texto.
- **Capa BiLSTM:** La capa principal del modelo corresponde a una red BiLSTM bidireccional, compuesta por neuronas recurrentes capaces de procesar simultáneamente la secuencia textual en dirección hacia adelante y hacia atrás. Esta característica permite capturar dependencias contextuales tanto previas como posteriores dentro del texto, favoreciendo una comprensión más completa del contenido semántico de la noticia.
- **Capa Dropout:** Se incorpora una capa de regularización *Dropout* para reducir el riesgo de sobreajuste, desactivando aleatoriamente un porcentaje de neuronas durante el entrenamiento y promoviendo una mejor generalización del modelo.
- **Capa de salida:** Finalmente, la arquitectura utiliza una capa densa de salida con una única neurona y función de activación sigmoideal, diseñada para generar una probabilidad continua entre 0 y 1 correspondiente a la clasificación binaria del problema:
 - Valor cercano a **0**: noticia verdadera.
 - Valor cercano a **1**: noticia falsa.

Configuración de hiperparámetros principales

Parámetro	Valor
Longitud máxima de secuencia	500 tokens
Dimensión Embedding	128
Unidades BiLSTM	64
Dropout	0.3
Función de activación salida	Sigmoid
Función de pérdida	Binary Crossentropy
Optimizador	Adam
Batch size	32
Epochs	10–15

f. Elaboración de modelos

En esta etapa se implementaron dos enfoques de clasificación: modelos clásicos basados en frecuencia de términos y modelos de aprendizaje profundo basados en secuencias.

Modelos Clásicos (Baseline):

- **Vectorización:** Se utilizó TfidfVectorizer con un límite de 15,000 características y eliminación de *stop words* en inglés.
- **Algoritmos:** Se entrenaron y evaluaron Naive Bayes (MultinomialNB), Regresión Logística y Máquinas de Vectores de Soporte Lineal (LinearSVC).

Modelos de Deep Learning:

- **Preprocesamiento:** Los textos se convirtieron en secuencias de enteros mediante un Tokenizer y se ajustaron a una longitud máxima de 300 palabras mediante *padding*.
- **Arquitectura BiLSTM:** Se diseñó una red neuronal con una capa de Embedding de 128 dimensiones, seguida de una capa Bidireccional LSTM con 64 unidades para capturar el contexto en ambos sentidos del texto. Se incluyó una capa de Dropout (0.3) para evitar el sobreajuste y una neurona de salida con función Sigmoid.
- **Entrenamiento:** Se empleó el optimizador Adam y la función de pérdida `binary_crossentropy`, utilizando un mecanismo de EarlyStopping para detener el proceso al alcanzar la convergencia óptima.

Pipeline de Procesamiento:

1. **Tokenización y Padding:** Los textos se convirtieron en vectores numéricos. Se definió un `max_length` de 300 palabras. Si la noticia era más corta, se rellenó con ceros (`padding='post'`); si era más larga, se recortó. Esto garantiza que la red reciba siempre una entrada de tamaño constante.
2. **Capa de Embedding (Incrustación):** Primera capa de la red. Su función es convertir cada palabra en un vector de 128 dimensiones en un espacio continuo, donde palabras con significados similares (ej. "mentira" y "engaño") quedan posicionadas cerca una de la otra.
3. **Regularización (Dropout):** Para evitar que el modelo se memorice los datos de entrenamiento (overfitting), se aplicó una capa de Dropout del 30% (0.3). Esto obliga a la red a aprender patrones generales en lugar de detalles específicos del ruido de los datos.
4. **Función de Activación:** En la última capa se usó Sigmoid, que comprime la salida a un rango entre 0 y 1, lo que permite interpretar el resultado como la "probabilidad de que la noticia sea real".

g. Desarrollo frontend y backend.

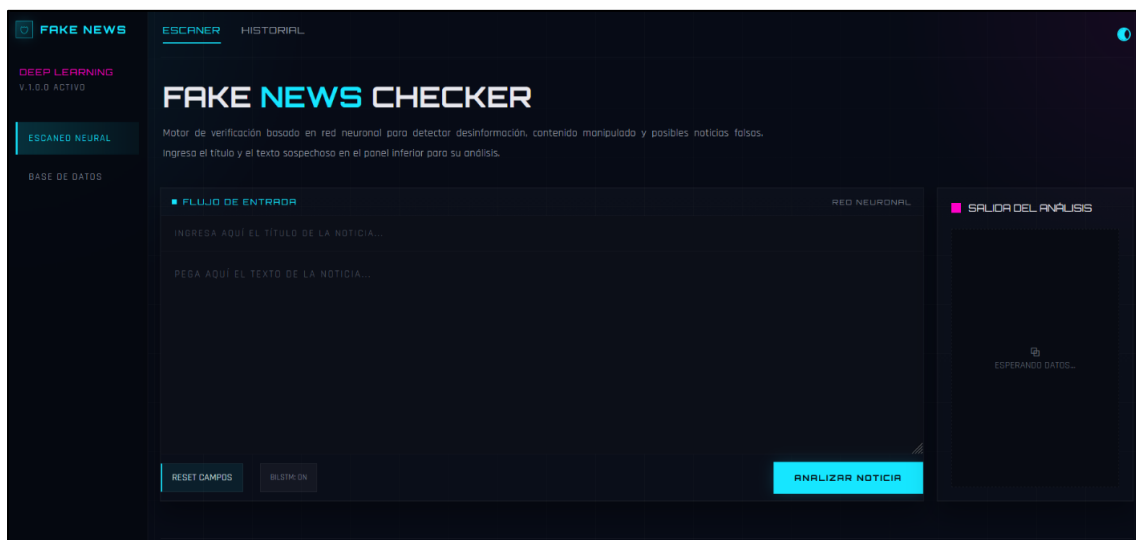
En esta etapa se consolidó la implementación funcional del sistema, centrada en la construcción de la interfaz web y su conexión con el backend encargado de ejecutar las predicciones. Este avance permitió llevar a la práctica los requerimientos relacionados con el ingreso de noticias, el envío de la información para su análisis y la presentación del resultado al usuario.

La aplicación web fue desarrollada en Laravel, utilizando Blade Templates y CSS, con el fin de ofrecer una experiencia clara, ordenada y fácil de utilizar. En esta fase se construyó la pantalla principal desde la cual el usuario puede ingresar el título y el contenido de una noticia para su análisis.

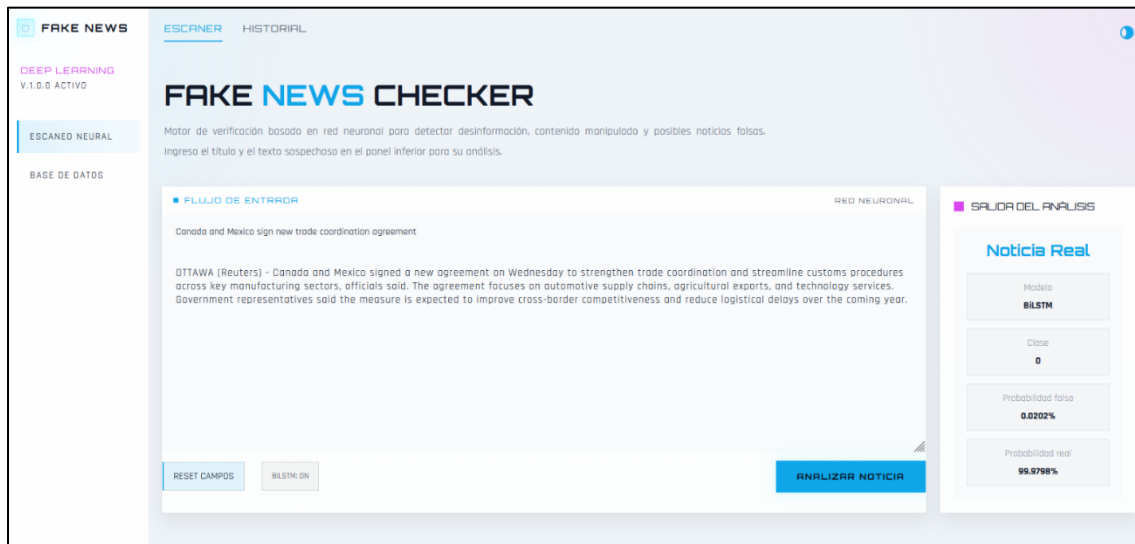
En paralelo, el backend fue implementado mediante una API en Flask, responsable de recibir la información enviada desde la interfaz y ejecutar la predicción con el modelo seleccionado, permitiendo una integración funcional entre la aplicación web y el sistema de clasificación.

Frontend

En el frontend se implementó la vista principal del sistema, desde la cual el usuario puede ingresar el título y el contenido de una noticia para su análisis. La interfaz incorpora soporte para tema claro y oscuro, manteniendo en ambos casos la misma estructura para el ingreso de datos, la ejecución del análisis y la visualización del resultado.

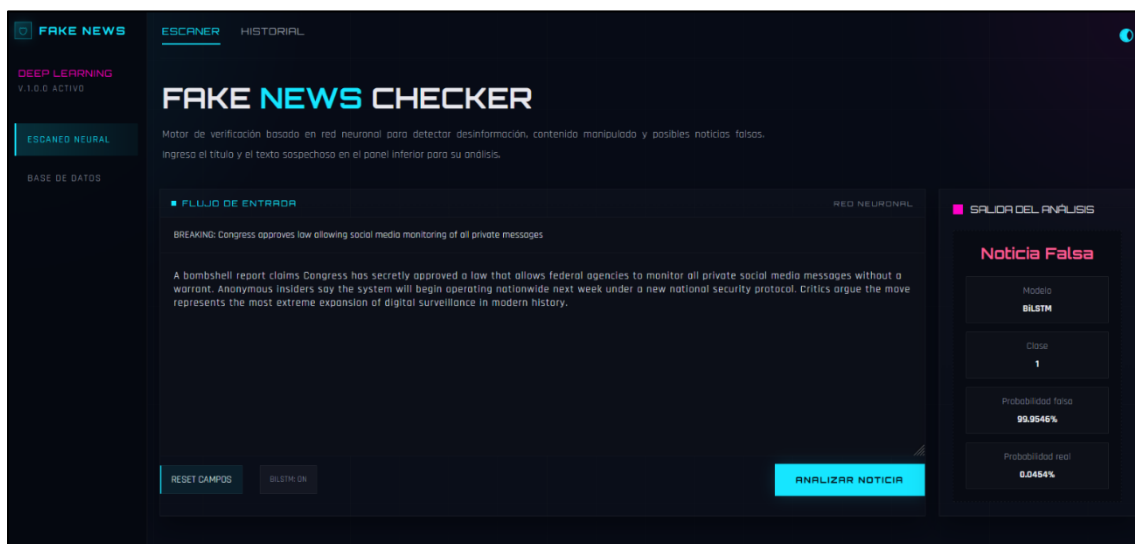


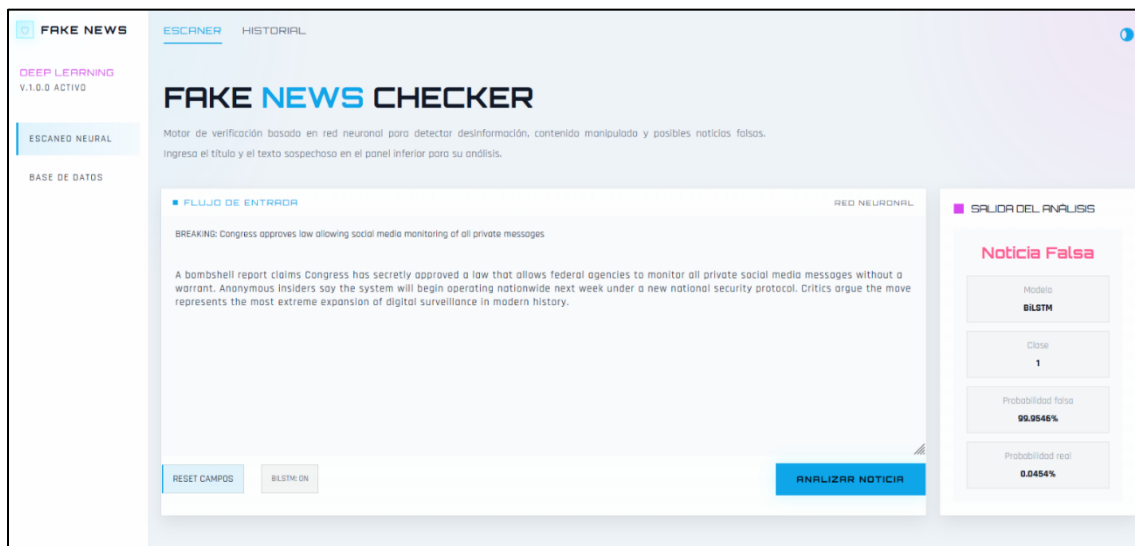
La sección “Salida del análisis” fue diseñada para indicar de manera comprensible si la noticia fue clasificada como real o falsa, junto con su nivel de confianza, mostrando la categoría predicha, el modelo utilizado y la probabilidad asociada a cada clase. En el ejemplo presentado, el sistema identifica la noticia como Noticia Real (clase 0) utilizando el modelo BiLSTM, con una probabilidad de 0.0202% para la clase falsa y 99.9798% para la clase real.



También se realizaron pruebas con una noticia clasificada como falsa. En este caso, la sección de salida indica Noticia Falsa (clase 1) mediante el modelo BiLSTM, con una probabilidad de 99.9546% para la clase falsa y 0.0454% para la clase real, lo que demuestra que la aplicación web presenta correctamente los mensajes asociados a cada clasificación.

El menú de navegación incorpora además las opciones Historial y Base de datos, correspondientes a vistas previstas para etapas posteriores del desarrollo, las cuales permitirán almacenar y consultar noticias previamente analizadas junto con su título, contenido, clasificación, probabilidades y fecha de ejecución.





Backend

En el backend se implementó una API en Flask encargada de recibir los datos enviados desde la interfaz web, procesarlos y retornar la respuesta con la predicción. Esto permitió conectar la aplicación desarrollada en Laravel con el modelo de clasificación. Como primera validación, se comprobó desde el navegador que la API respondiera correctamente en entorno local. Luego, se verificó la carga del modelo BiLSTM, del tokenizer y de los parámetros necesarios para la predicción, confirmando que el servicio se encuentra operativo y disponible para recibir solicitudes desde la aplicación web.

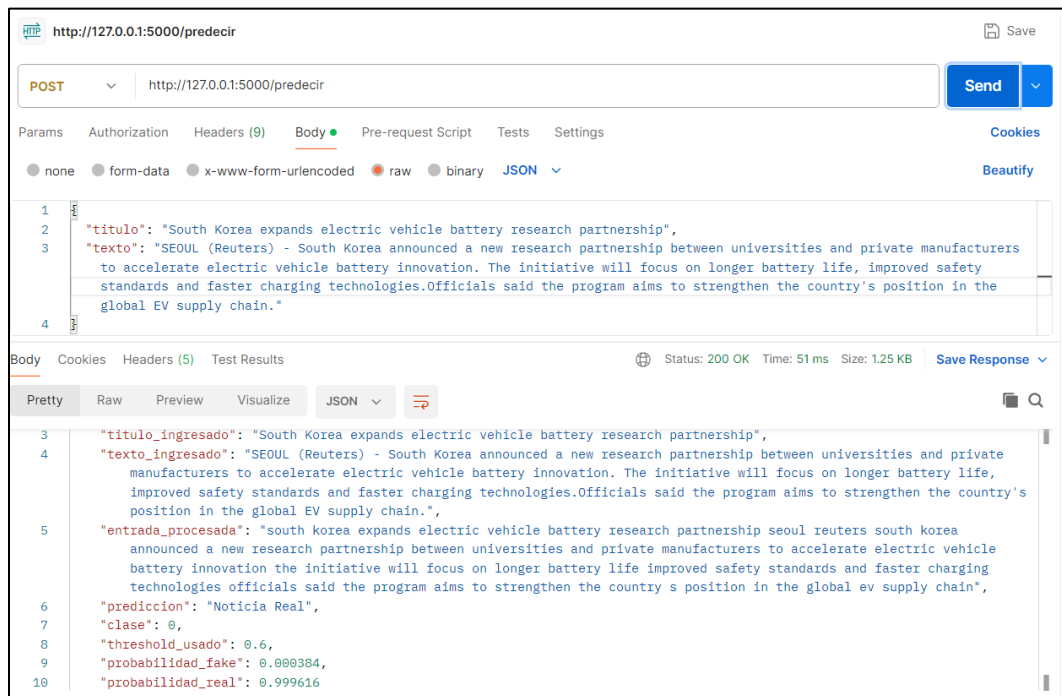


```

Modelo, tokenizer, max_length y threshold cargados correctamente.
Modelo cargado: BiLSTM
* Serving Flask app 'fake_news_api'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://192.168.100.5:5000
Press CTRL+C to quit
127.0.0.1 - - [11/Apr/2026 20:37:47] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [11/Apr/2026 20:37:47] "GET /favicon.ico HTTP/1.1" 404 -

```

De forma complementaria, se realizaron pruebas directas sobre la API mediante Postman, herramienta que permitió validar el correcto envío de solicitudes y la estructura de las respuestas entregadas por el backend.



V. Resultados

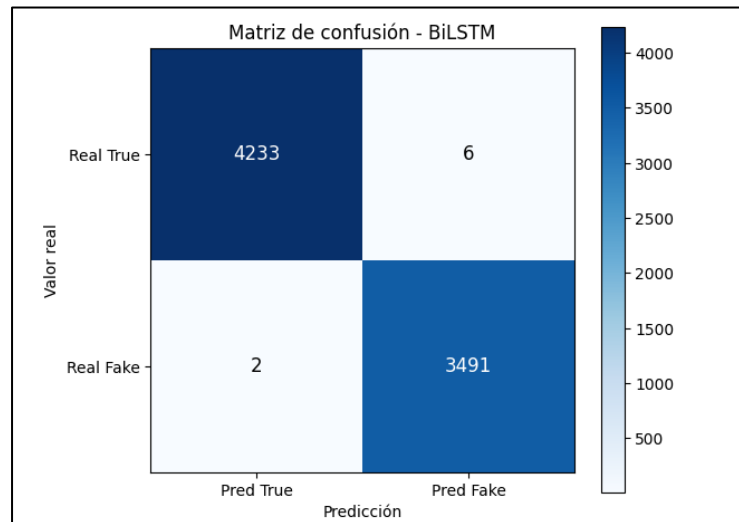
La evaluación de los modelos se realizó sobre un conjunto de prueba independiente (20% del dataset), obteniendo un rendimiento excepcionalmente alto en la distinción de noticias.

	Modelo	Tipo	Accuracy	Precision	Recall	F1-score	Threshold
0	BiLSTM	Deep Learning	0.998965	0.998284	0.999427	0.998856	0.60
1	CNN 1D	Deep Learning	0.998060	0.996857	0.998855	0.997855	0.59
2	LinearSVC	Machine Learning	0.992499	0.995957	0.987403	0.991662	NaN
3	LogisticRegression	Machine Learning	0.983833	0.988116	0.975952	0.981996	NaN
4	LSTM	Deep Learning	0.981376	0.973156	0.985972	0.979522	0.81
5	MultinomialNB	Machine Learning	0.935334	0.931890	0.924420	0.928140	NaN

Los valores de BiLSTM consolidan el mejor rendimiento global según las pruebas realizadas en el notebook de modelado.

Análisis de resultados:

- El modelo BiLSTM superó ligeramente a los modelos clásicos y a la CNN 1D, demostrando que la comprensión bidireccional de las secuencias de texto es clave para identificar patrones de desinformación.
- La matriz de confusión muestra un equilibrio sólido entre las clases, con una tasa mínima de falsos positivos y negativos, lo que garantiza la confiabilidad requerida para el sistema (RNF10).



El modelo BiLSTM mostró un desempeño superior con un F1-score de 99.88%. La matriz de confusión revela que el modelo clasificó correctamente casi la totalidad de las noticias de prueba, con una cantidad mínima de falsos positivos y falsos negativos, lo que valida su uso para la detección de desinformación.

El modelo convergió rápidamente (en pocas épocas). Gracias al uso de EarlyStopping, el entrenamiento se detuvo en el momento justo cuando el error de validación dejó de bajar, asegurando que el modelo sea capaz de generalizar con noticias nuevas que nunca ha visto.

Al observar la matriz, se identifica que el modelo tiene un equilibrio casi perfecto. La baja tasa de Falsos Negativos (noticias falsas clasificadas como reales) es vital para el éxito del proyecto, ya que el objetivo principal es que ninguna desinformación pase desapercibida.

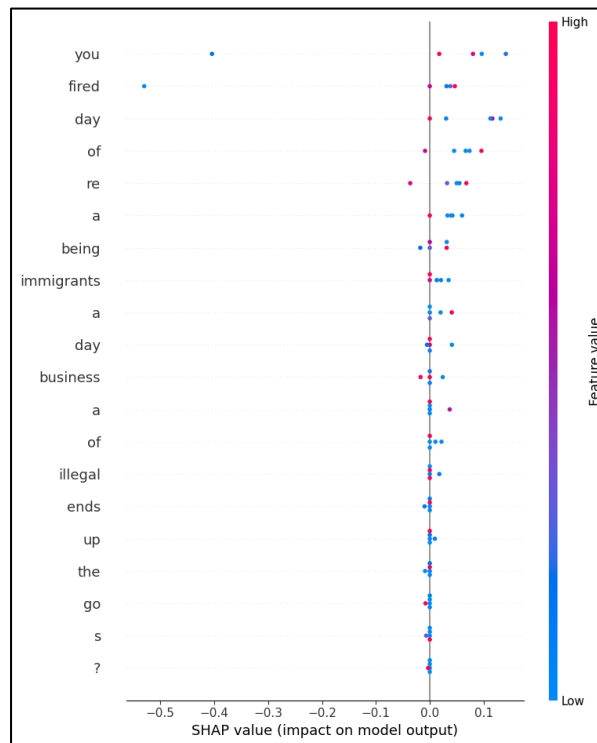
Se Resalta que los resultados son confiables porque se probaron sobre una base de datos de casi 39,000 registros, lo que da significancia estadística con 99% de precisión.

1. Análisis SHAP

Gráfico de Resumen (Summary Plot) tipo "Beeswarm" (enjambre de abejas) de SHAP. Muestra cada noticia individualmente como un punto.

El gráfico de tipo *summary plot* basado en valores SHAP permite identificar la contribución de

cada variable (en este caso, palabras) sobre la salida del modelo. En el eje vertical se presentan las variables ordenadas según su importancia global, mientras que el eje horizontal representa el impacto de cada variable en la predicción, donde valores positivos incrementan la salida del modelo y valores negativos la disminuyen.



En este caso, las variables corresponden a palabras individuales, lo que indica que el modelo está basado en procesamiento de lenguaje natural (NLP). A partir del análisis del gráfico, se observa que las palabras con mayor influencia en la predicción son “you”, “fired”, “day”, “of”, “re”, “being” e “immigrants”, ubicadas en la parte superior del gráfico.

Las palabras “you” y “fired” presentan una alta dispersión en sus valores SHAP, incluyendo contribuciones negativas significativas. Esto sugiere que su impacto en la predicción es altamente dependiente del contexto en el que aparecen, pudiendo tanto aumentar como disminuir la salida del modelo.

Por otro lado, la palabra “immigrants” muestra principalmente contribuciones positivas, lo que indica que su presencia tiende a incrementar la predicción del modelo. De manera similar, la palabra “illegal” también presenta un efecto positivo, aunque de menor magnitud. Este comportamiento sugiere que el modelo asocia estos términos con la clase objetivo, lo que podría reflejar patrones aprendidos del conjunto de datos.

En contraste, palabras comunes como “day”, “of”, “a” y “the” presentan valores SHAP cercanos a cero, lo que indica que tienen una contribución marginal en la predicción. Esto es consistente con su rol como palabras funcionales (stopwords), que generalmente no aportan información semántica relevante.

Asimismo, se observa que varias palabras ubicadas en la parte inferior del gráfico tienen una influencia prácticamente nula, lo que refuerza la idea de que el modelo se apoya principalmente en un subconjunto reducido de términos con mayor carga semántica.

En términos generales, el modelo parece basar sus decisiones en la presencia de palabras específicas con alto contenido informativo, mientras que ignora en gran medida palabras de uso común. Sin embargo, la fuerte influencia de ciertos términos como “immigrants” e “illegal” sugiere la posible existencia de sesgos en el modelo, lo cual debería ser considerado en su evaluación y validación.

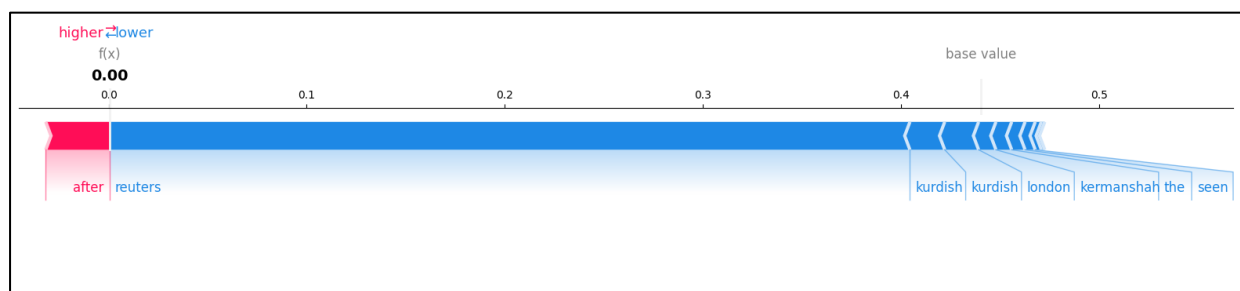
En conclusión, el análisis del summary plot de SHAP permite evidenciar qué variables influyen en mayor medida en la predicción del modelo, así como la dirección de dicho impacto, proporcionando una herramienta útil para la interpretabilidad y diagnóstico del comportamiento del modelo.

2. Análisis del Force Plot de SHAP

El siguiente gráfico presentado corresponde a un *force plot* de SHAP, el cual permite visualizar cómo cada variable contribuye a una predicción individual del modelo. A diferencia del *summary plot*, este tipo de gráfico explica una sola observación, mostrando cómo se pasa desde un valor base (*base value*) hasta la predicción final ($f(x)$).

En el gráfico, el valor base representa la predicción promedio del modelo, mientras que el valor final ($f(x) = 0.00$) corresponde a la salida específica para la instancia analizada. La diferencia entre ambos valores es explicada por la contribución de cada variable.

- La proporción de cada color en la barra indica cuánto contribuye esa palabra a cada clase.



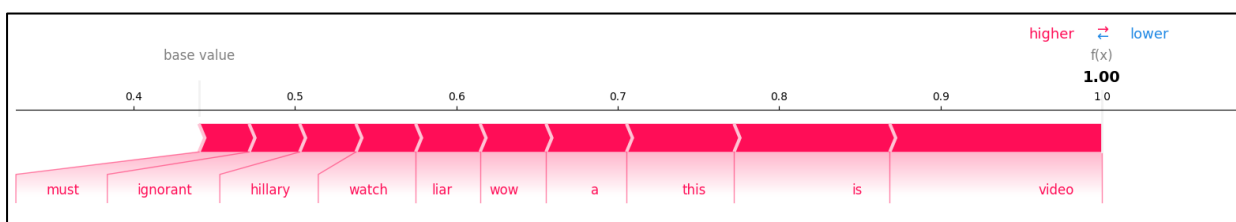
Ejemplo Clave: "reuters". Es una de las barras más largas. En este dataset, casi todas las noticias reales incluyen la palabra "reuters" al inicio. Por eso, el modelo la usa como una "señal" casi inequívoca de que la noticia es **Verdadera (Clase 0)**.

Las variables que aparecen en color rojo corresponden a aquellas que aumentan la predicción, mientras que las variables en color azul la disminuyen. En este caso, se observa que la mayoría de las palabras contribuyen negativamente (en azul), empujando la predicción desde el valor base hacia un valor cercano a cero.

Entre las palabras con mayor impacto negativo se encuentran “reuters”, “kurdish”, “london”, “kermanshah”, “the” y “seen”. Estas variables son responsables de reducir significativamente la salida del modelo, lo que indica que su presencia está asociada a una menor probabilidad de la clase objetivo.

Por otro lado, se observa una contribución positiva menor (en rojo), destacando la palabra “after”, la cual incrementa levemente la predicción. Sin embargo, su efecto es considerablemente inferior en comparación con las contribuciones negativas.

En términos generales, el modelo parece interpretar esta instancia como perteneciente a una clase con baja probabilidad, dado que la mayoría de las variables relevantes empujan la predicción hacia valores bajos. Además, el predominio de contribuciones negativas sugiere que las características presentes en el texto no coinciden con los patrones que el modelo asocia con la clase positiva.



Ejemplo Clave: "video", "wow", "watch". Estas palabras suelen aparecer mucho en los titulares de noticias falsas (ej. "WATCH: [VIDEO]..."). Por tanto, el modelo las asocia con la **Clase 1 (Fake)**.

En este caso, el valor final de la predicción es $f(x) = 1.00$, lo que indica que el modelo asigna una probabilidad máxima a la clase objetivo. El valor base, ubicado aproximadamente en torno a 0.45–0.50, representa la predicción promedio del modelo antes de considerar las variables de la instancia.

A diferencia del caso anterior, se observa que todas las variables relevantes contribuyen de manera positiva (representadas en color rojo), empujando la predicción hacia el extremo superior. Esto indica que el conjunto de características presentes en esta instancia coincide fuertemente con los patrones que el modelo asocia con la clase positiva.

Entre las variables con mayor impacto se encuentran las palabras “must”, “ignorant”, “hillary”, “watch”, “liar”, “wow”, “this”, “is” y “video”. Estas palabras contribuyen acumulativamente a aumentar la predicción, siendo responsables del desplazamiento desde el valor base hasta el valor final de 1.00.

Es particularmente relevante la presencia de términos como “ignorant” y “liar”, que poseen una fuerte carga semántica negativa. Esto sugiere que el modelo ha aprendido a asociar este tipo de lenguaje con la clase objetivo (por ejemplo, contenido ofensivo o polarizado, dependiendo de la tarea). Asimismo, la inclusión de “hillary” indica que el modelo también puede estar capturando contextos relacionados con figuras públicas o temas políticos.

El hecho de que todas las contribuciones sean positivas y no se observen efectos negativos indica una alta coherencia entre las variables de entrada y la predicción final. En otras palabras, no existen elementos en la instancia que contrarresten la decisión del modelo.

Como la palabra **"reuters"** tiene un impacto masivo, significa que el modelo aprendió que "si dice Reuters, es verdad". Esto es muy preciso para este data set, pero en el "mundo real" (fuera de este dataset), una noticia falsa podría simplemente escribir la palabra "Reuters" para engañar al modelo. Esto se conoce como un sesgo de la data set que SHAP permite identificar con claridad.

3. Relación entre el uso de SHAP y el cumplimiento de requerimientos del sistema

En relación con los requerimientos funcionales, el uso de SHAP permite evidenciar el cumplimiento del RF04 (Procesamiento y comparación), ya que los valores SHAP muestran explícitamente cómo el modelo evalúa las características lingüísticas del texto y las compara con los patrones aprendidos durante el entrenamiento. Esto se observa en la identificación de palabras que influyen positiva o negativamente en la predicción, reflejando el proceso interno de análisis del modelo.

Asimismo, se cumple el RF05 (Clasificación de veracidad), dado que los *force plots* permiten visualizar la predicción final del modelo junto con su valor asociado de probabilidad. Esto facilita la interpretación del resultado, mostrando cómo cada característica contribuye a la decisión final, ya sea clasificando la noticia como verdadera o falsa.

Respecto a los requerimientos no funcionales, el uso de SHAP resulta fundamental para el cumplimiento del RNF05 (Explicabilidad). Estas técnicas permiten proporcionar una justificación clara de las predicciones del modelo, identificando las palabras o patrones más relevantes que influyen en la clasificación. De este modo, el sistema no se limita a emitir un resultado, sino que también entrega información interpretativa que facilita la comprensión, reduciendo la desconfianza en sistemas automatizados.

Además, SHAP contribuye al RNF03 (Confiabilidad), ya que permite analizar el comportamiento del modelo frente a distintos tipos de entradas, facilitando la detección de posibles inconsistencias o sesgos. Esto fortalece la robustez del sistema al permitir ajustes informados.

En relación con el RNF07 (Monitoreabilidad), los valores SHAP pueden ser utilizados como una herramienta de diagnóstico continuo, permitiendo observar cambios en la importancia de las variables y detectar desviaciones en el comportamiento del modelo cuando se enfrenta a nuevos datos.

Por otro lado, el cumplimiento del RNF04 (Usabilidad) depende de la adecuada integración de estas visualizaciones en la interfaz de usuario. Si bien SHAP proporciona información valiosa, su complejidad requiere ser presentada de manera simplificada para garantizar su comprensión por parte de usuarios no técnicos.

Finalmente, aunque SHAP no incide directamente en el rendimiento del modelo, contribuye indirectamente al RNF10 (Calidad del modelo), ya que permite identificar errores, sesgos o dependencias excesivas en ciertas variables, facilitando la mejora continua del sistema.

VI. Propuesta de mejoras

A partir de los resultados obtenidos y las limitaciones identificadas en el desarrollo, se proponen las siguientes mejoras para versiones futuras del sistema:

1. **Implementación de Modelos basados en Transformers:** Migrar hacia arquitecturas como BERT o RoBERTa. Aunque la BiLSTM tuvo un desempeño excelente, los Transformers podrían captar matices semánticos más profundos y reducir aún más la ambigüedad en noticias complejas.
2. **Dataset Multilingüe y Diversificado:** Ampliar el entrenamiento a noticias en español y otros idiomas. Actualmente, el sistema está optimizado para inglés; incluir datos locales permitiría una mayor aplicabilidad en el contexto regional.
3. **Optimización del Pipeline de Inferencia:** Implementar un sistema de caché o una cola de procesamiento en la API de Flask para mejorar los tiempos de respuesta cuando el volumen de consultas aumente, asegurando que se mantenga por debajo de los 2 segundos definidos en los requerimientos (RNF01).

VII. Código de fuente en GitHub (Anexo link).

<https://github.com/AlejandroDart/fake-news-system>

VIII. Conclusión

El desarrollo del presente proyecto permitió abordar de manera integral la problemática de la detección automática de noticias falsas mediante la aplicación de técnicas de aprendizaje de máquina y aprendizaje profundo, demostrando que el uso de modelos de clasificación supervisada constituye una alternativa técnicamente viable, eficiente y escalable para enfrentar uno de los principales desafíos del ecosistema digital contemporáneo: la propagación masiva de desinformación en medios digitales y redes sociales.

A lo largo del trabajo se logró diseñar, implementar y validar una solución funcional orientada a la clasificación binaria de noticias, integrando tanto componentes de ciencia de datos como de desarrollo de software. Desde una perspectiva metodológica, la utilización del enfoque híbrido basado en CRISP-DM y Scrum permitió estructurar ordenadamente cada fase del proyecto, facilitando la comprensión del problema, la exploración y preparación de datos, el entrenamiento y evaluación de modelos, así como la construcción incremental de la arquitectura tecnológica del sistema.

En la etapa experimental se evaluaron múltiples técnicas de clasificación, incluyendo modelos clásicos de aprendizaje automático y arquitecturas de aprendizaje profundo, lo que permitió establecer una comparación objetiva entre distintos enfoques de complejidad creciente. Los resultados obtenidos evidenciaron que, si bien los modelos tradicionales presentan un buen desempeño inicial y una implementación computacionalmente eficiente, la arquitectura BiLSTM destacó como la alternativa más robusta dentro de los experimentos realizados, debido a su capacidad superior para modelar dependencias contextuales complejas y relaciones semánticas presentes en textos periodísticos. Esta arquitectura logró satisfacer satisfactoriamente los requerimientos de desempeño definidos para el proyecto, alcanzando métricas superiores al umbral esperado y demostrando un comportamiento estable durante el entrenamiento y la validación.

Adicionalmente, uno de los principales aportes del proyecto radica en haber trascendido la fase puramente experimental de entrenamiento de modelos, desarrollando una implementación práctica completamente funcional mediante una arquitectura modular compuesta por frontend web, backend API y motor de inferencia integrado. Esta implementación permitió validar que el modelo seleccionado no sólo posee un desempeño adecuado en entornos controlados de entrenamiento, sino también viabilidad operativa en escenarios de uso real, siendo capaz de procesar entradas desde una interfaz de usuario y generar predicciones en tiempo real dentro de tiempos de respuesta compatibles con los requerimientos establecidos.

No obstante, durante el desarrollo también se identificaron limitaciones relevantes asociadas al alcance de la solución propuesta. Entre ellas destaca la dependencia respecto de la calidad y representatividad del conjunto de datos utilizado, así como la posible existencia de sesgos temáticos o editoriales dentro del dataset, factores que pueden influir en la generalización del modelo hacia contextos reales más diversos. Asimismo, debe considerarse que la solución desarrollada no busca establecer una verdad absoluta respecto de una noticia, sino entregar una estimación probabilística basada en patrones previamente aprendidos, funcionando como una

herramienta de apoyo preliminar para la detección de posibles contenidos engañosos.

En términos generales, puede concluirse que el presente proyecto cumplió satisfactoriamente con los objetivos planteados inicialmente, logrando desarrollar una solución tecnológica completa, fundamentada metodológica y técnicamente robusta, capaz de clasificar noticias como verdaderas o falsas mediante el uso de inteligencia artificial aplicada al procesamiento de lenguaje natural. De esta manera, el trabajo realizado no sólo aporta evidencia práctica sobre la efectividad del aprendizaje profundo en problemas de clasificación textual compleja, sino que también establece una base sólida para futuras mejoras, ampliaciones y líneas de investigación orientadas al perfeccionamiento continuo de sistemas automatizados de detección de desinformación.

Finalmente, este proyecto demuestra que la combinación entre técnicas avanzadas de inteligencia artificial, arquitecturas modernas de software y metodologías estructuradas de desarrollo constituye una estrategia altamente efectiva para abordar problemas reales de impacto social, posicionando al aprendizaje automático como una herramienta clave dentro del desarrollo de soluciones tecnológicas innovadoras aplicadas al análisis inteligente de información digital.

Futuras líneas de trabajo podrían orientarse a la incorporación de modelos basados en Transformers, mecanismos de explicabilidad (XAI) y datasets multilingües de mayor diversidad temática.

IX. Bibliografía

Amorós García, M. (2018). *Fake news: La verdad de las noticias falsas*. Plataforma Editorial.

Beauvais, C. (2022). Fake news: Why do we believe it? *Joint Bone Spine*, 89(4), 105371. <https://doi.org/10.1016/j.jbspin.2022.105371>

Broda, E., & Strömbäck, J. (2024). Misinformation, disinformation, and fake news: Lessons from an interdisciplinary, systematic literature review. *Annals of the International Communication Association*, 48(2), 139–166.

Capuano, N., Fenza, G., Loia, V., & Nota, F. D. (2023). Content-based fake news detection with machine and deep learning: A systematic review. *Neurocomputing*, 530, 91–103. <https://doi.org/10.1016/j.neucom.2023.02.005>

Emil, R. Ş., & Remus, B. (2025). A review of automatic fake news detection: From traditional methods to large language models. *Future Internet*, 17(10), 435. <https://doi.org/10.3390/fi17100435>

Fedushko, S., Syerov, Y., & Kryvinska, N. (2024). AntiFake system: Machine learning-based system for verification of fake news. *Procedia Computer Science*, 238, 663–670. <https://doi.org/10.1016/j.procs.2024.06.075>

García-Marín, D. (2022). Fake news y posverdad: relación con las redes sociales y fiabilidad de contenidos. *Estudios sobre el Mensaje Periodístico*, 28(3), 483–493.

Hu, B., Mao, Z., & Zhang, Y. (2025). An overview of fake news detection: From a new perspective. *Fundamental Research*, 5(1), 332–346. <https://doi.org/10.1016/j.fmre.2024.01.017>

Imbwaga, J. L., Chittaragi, N., & Koolagudi, S. G. (2022). Fake news detection using machine learning algorithms. En *Proceedings of the 2022 Fourteenth International Conference on Contemporary Computing (IC3-2022)* (pp. 271–275). Association for Computing Machinery. <https://doi.org/10.1145/3549206.3549256>

Passi, K., & Shah, A. (2022). Distinguishing fake and real news of Twitter data with the help of machine learning techniques. In *Proceedings of the 26th International Database Engineered Applications Symposium* (pp. 1–8). Association for Computing Machinery. <https://doi.org/10.1145/3548785.3548811>

Rojas-Rubio, L., & Meneses-Villegas, C. (2022). Una comparación empírica de algoritmos de aprendizaje automático versus aprendizaje profundo para la detección de noticias falsas en redes sociales. *Ingeniare. Revista Chilena de Ingeniería*, 30(2), 418–431. <https://doi.org/10.4067/S0718-33052022000200403>

Sudhakar, M., & Kaliyamurthi, K. P. (2024). Detection of fake news from social media using support vector machine learning algorithms. *Measurement: Sensors*, 32, 101028. <https://doi.org/10.1016/j.measen.2024.101028>